

Java 8 Revision notes

1. Lambda Expressions

- Introduced to enable functional programming in Java.
- Lambda expression is an anonymous function (no name, no return type, no access modifier)

Syntax:

```
(parameters) -> expression
(parameters) -> { statements; }
```

- **Examples:**

```
// Without Lambda
Runnable r1 = new Runnable() {
    @Override
    public void run() {
        System.out.println("Without Lambda");
    }
};
r1.run();

// With Lambda
Runnable r2 = () -> System.out.println("With Lambda");
r2.run();
```

- **Common Use Cases:**

- Iterating collections:

```
List<String> names = Arrays.asList("John", "Jane", "Jack");
names.forEach(name -> System.out.println(name));
```

- Sorting:

```
List<String> names = Arrays.asList("John", "Jane", "Jack");
names.sort((a, b) -> a.compareTo(b));
```

2. Functional Interfaces

- An interface with exactly one abstract method.
- **Annotation:** `@FunctionalInterface` (optional but recommended).
- **Built-in Functional Interfaces (from `java.util.function`):**

Interface	Description	Example
<code>Predicate<T></code>	Tests a condition (<code>true</code> / <code>false</code>).	<code>x -> x > 10</code>
<code>Function<T, R></code>	Transforms <code>T</code> into <code>R</code> .	<code>x -> x.toUpperCase()</code>
<code>Consumer<T></code>	Performs an action on <code>T</code> .	<code>x -> System.out.println(x)</code>
<code>Supplier<T></code>	Supplies a value (no input).	<code>() -> new Date()</code>
<code>BinaryOperator<T></code>	Combines two <code>T</code> into one <code>T</code> .	<code>(x, y) -> x + y</code>
<code>UnaryOperator<T></code>	Applies operation on a single <code>T</code> .	<code>x -> x * x</code>

Lets look at the example in the program below:

```
import java.util.Arrays;
import java.util.List;
```

```
import java.util.function.*;
import java.util.stream.Collectors;

class MobilePhone {
    String name;

    public MobilePhone(String name) {
        this.name = name;
    }
}

//-----
@FunctionalInterface
interface MathOperation {
    int operate(int a, int b);
}

// -----

public class Java8Demo {
    public static void main(String[] args) {

        MathOperation sumOperation = (a, b) -> a + b;
        MathOperation subtractOperation = (a, b) -> a - b;
        int res = sumOperation.operate(1, 2);
        System.out.println(res);
        // Output: 3

        // Predicate --> Functional interface (Boolean valued function)
        Predicate<Integer> isEven = x -> x % 2 == 0;
        System.out.println(isEven.test(4));
        // Output: true

        Predicate<String> isWordStartingWithA = x -> x.toLowerCase().startsWith("a");
        Predicate<String> isWordEndingWithT = x -> x.toLowerCase().endsWith("t");
        Predicate<String> and = isWordStartingWithA.and(isWordEndingWithT);
        System.out.println(and.test("Akshay"));
        // Output: false (does not end with 't')

        // Function --> work for you
        Function<Integer, Integer> doubleIt = x -> 2 * x;
        Function<Integer, Integer> tripleIt = x -> 3 * x;

        System.out.println(doubleIt.andThen(tripleIt).apply(20));
        // Output: 120 (20 * 2 = 40, 40 * 3 = 120)

        System.out.println(tripleIt.andThen(doubleIt).apply(20));
        // Output: 120 (20 * 3 = 60, 60 * 2 = 120)

        System.out.println(doubleIt.compose(tripleIt).apply(20));
        // Output: 120 (20 * 3 = 60, 60 * 2 = 120)

        System.out.println(doubleIt.apply(100));
        // Output: 200 (100 * 2 = 200)

        Function<Integer, Integer> identity = Function.identity();
        Integer res2 = identity.apply(5);
```

```

System.out.println(res2);
// Output: 5

// Consumer
Consumer<Integer> print = x -> System.out.println(x);
print.accept(51);
// Output: 51

List<Integer> list = Arrays.asList(1, 2, 3);
Consumer<List<Integer>> printList = x -> {
    for (int i : x) {
        System.out.println(i);
    }
};
printList.accept(list);
// Output:
// 1
// 2
// 3

// Supplier
Supplier<String> giveHelloWorld = () -> "Hello World";
System.out.println(giveHelloWorld.get());

// Output: Hello World

// Combined example
Predicate<Integer> predicate = x -> x % 2 == 0;
Function<Integer, Integer> function = x -> x * x;
Consumer<Integer> consumer = x -> System.out.println(x);
Supplier<Integer> supplier = () -> 100;

if (predicate.test(supplier.get())) {
    consumer.accept(function.apply(supplier.get()));
}
// Output: 10000 (square of 100)

// BiPredicate, BiConsumer, BiFunction
BiPredicate<Integer, Integer> isSumEven = (x, y) -> (x + y) % 2 == 0;
System.out.println(isSumEven.test(5, 5));

// Output: true (5 + 5 = 10, which is even)

BiConsumer<Integer, String> biConsumer = (x, y) -> {
    System.out.println(x);
    System.out.println(y);
};
biConsumer.accept(10, "Hello");
// Output:
// 10
// Hello

BiFunction<String, String, Integer> biFunction = (x, y) -> (x + y).length();
System.out.println(biFunction.apply("a", "bc"));
// Output: 3 (length of "abc")

// UnaryOperator, BinaryOperator
UnaryOperator<Integer> a = x -> 2 * x;

```

```

BinaryOperator<Integer> b = (x, y) -> x + y;

System.out.println(a.apply(5));
// Output: 10

System.out.println(b.apply(5, 5));
// Output: 10

// Method reference --> use method without invoking & in place of lambda expressio
n

List<String> students = Arrays.asList("Ram", "Shyam", "Ghanshyam");
students.forEach(x -> System.out.println(x));
students.forEach(System.out::println);
// Output:
// Ram
// Shyam
// Ghanshyam

// Constructor reference
List<String> names = Arrays.asList("A", "B", "C");
List<MobilePhone> mobilePhoneList = names.stream().map(MobilePhone::new).collect(C
ollectors.toList());

// No direct output here; mobilePhoneList contains MobilePhone objects with names
"A", "B", "C"
}
}

```

3. Streams API

- **What is a Stream?**

A sequence of elements supporting aggregate operations like `filter`, `map`, `reduce`.

- **Stream Pipeline:**

1. **Source:** Generate a stream (e.g., `Collection.stream()`, `Stream.of()`).
2. **Intermediate Operations:** Transform the stream (`filter`, `map`, `sorted`).
3. **Terminal Operations:** Consume the stream (`collect`, `forEach`, `reduce`).

1. Stream Creation

- **From Collection:** `stream()`, `parallelStream()`
- **From Array:** `Stream.of(array)`, `Arrays.stream(array)`
- **From Range:** `IntStream.range(start, end)`, `IntStream.rangeClosed(start, end)`
- **From File:** `Files.lines(Paths.get("file.txt"))`
- **Generate/Iterate:** `Stream.generate(Supplier)`, `Stream.iterate(seed, UnaryOperator)`

2. Intermediate Operations

Filtering

- `filter(Predicate)` - Filters elements matching a condition.

Mapping

- `map(Function)` - Transforms each element.

- `flatMap(Function)` - Flattens nested structures.

Sorting

- `sorted()` - Natural order.
- `sorted(Comparator)` - Custom comparator.

Distinct

- `distinct()` - Removes duplicates.

Limit/Skip

- `limit(n)` - Keeps first `n` elements.
- `skip(n)` - Skips first `n` elements.

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Stream;

public class IntermediateOps {
    public static void main(String[] args) {
        // Intermediate operations transform a stream into another stream
        // They are lazy, meaning they don't execute until a terminal operation is invoked.

        // 1. filter
        List<String> list = Arrays.asList("Akshit", "Ram", "Shyam", "Ghanshyam", "Akshit");
        Stream<String> filteredStream = list.stream().filter(x -> x.startsWith("A"));
        // No output yet (lazy operation, no terminal operation)

        long res = list.stream().filter(x -> x.startsWith("A")).count();
        System.out.println(res);
        // Output: 2 (two elements "Akshit" start with "A")

        // 2. map
        Stream<String> stringStream = list.stream().map(String::toUpperCase);
        // No output yet (lazy operation, no terminal operation)

        // 3. sorted
        Stream<String> sortedStream = list.stream().sorted();
        // No output yet (lazy operation, no terminal operation)

        Stream<String> sortedStreamUsingComparator = list.stream().sorted((a, b) -> a.length() - b.length());
        // No output yet (lazy operation, no terminal operation)

        // 4. distinct
        System.out.println(list.stream().filter(x -> x.startsWith("A")).distinct().count());
        // Output: 1 (distinct removes duplicates; "Akshit" appears twice)

        // 5. limit
        System.out.println(Stream.iterate(1, x -> x + 1).limit(100).count());
        // Output: 100 (stream is limited to 100 elements starting from 1)

        // 6. skip
        System.out.println(Stream.iterate(1, x -> x + 1).skip(10).limit(100).count());
        // Output: 100 (skipped the first 10 elements, then limited to 100 elements)
```

```
// 7. peek
// Performs an action on each element as it is consumed.
Stream.iterate(1, x -> x + 1).skip(10).limit(100).peek(System.out::println).count();
// Output: Prints numbers from 11 to 110 (peek prints each element as it is processed)

// 8. flatMap
// Handle streams of collections, lists, or arrays where each element is itself a collection
// Flatten nested structures (e.g., lists within lists) so that they can be processed as a single sequence of elements
// Transform and flatten elements at the same time.
List<List<String>> listOfLists = Arrays.asList(
    Arrays.asList("apple", "banana"),
    Arrays.asList("orange", "kiwi"),
    Arrays.asList("pear", "grape")
);
System.out.println(listOfLists.get(1).get(1));
// Output: kiwi (accessing the second element of the second list)

System.out.println(listOfLists.stream().flatMap(x -> x.stream()).map(String::toUpperCase).toList());
// Output: [APPLE, BANANA, ORANGE, KIWI, PEAR, GRAPE]
// Flattens the nested list and converts all elements to uppercase

List<String> sentences = Arrays.asList(
    "Hello world",
    "Java streams are powerful",
    "flatMap is useful"
);
System.out.println(sentences
    .stream()
    .flatMap(sentence -> Arrays.stream(sentence.split(" ")))
    .map(String::toUpperCase)
    .toList());
// Output: [HELLO, WORLD, JAVA, STREAMS, ARE, POWERFUL, FLATMAP, IS, USEFUL]
// Splits each sentence into words, flattens them into a single stream, and converts all words to uppercase
}
```

This below program demonstrates **lazy evaluation** in streams:

- Intermediate operations like `filter()` are **not executed immediately**. Instead, they are evaluated only when a terminal operation (like `collect()`) is invoked.
- The actual filtering occurs **on demand** during the terminal operation.

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class LazyEvaluationDemo {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "David");
```

```

        Stream<String> stream = names.stream()
            .filter(name -> {
                System.out.println("Filtering: " + name);
                return name.length() > 3;
            });

        System.out.println("Before terminal operation");

        List<String> result = stream.collect(Collectors.toList());

        System.out.println("After terminal operation");
        System.out.println(result);
    }
}

```

```

Before terminal operation
Filtering: Alice
Filtering: Charlie
Filtering: David
After terminal operation
[Alice, Charlie, David]

```

3. Terminal Operations

Aggregation

- `count()` - Number of elements.
- `max(Comparator)` / `min(Comparator)` - Finds max/min element.

```
numbers.stream().max(Comparator.naturalOrder());
```

- `sum()` / `average()` - Available for numeric streams (`IntStream`, `DoubleStream`).

```
IntStream.of(1, 2, 3).sum(); // Output: 6
IntStream.of(1, 2, 3).average().orElse(0.0); // Output: 2.0
```

Reduction

- `reduce(BinaryOperator)` - Reduces elements into a single value.

```
numbers.stream().reduce(0, (a, b) -> a + b); // Sum of all elements
```

Matching

- `allMatch(Predicate)` - Checks if all elements match.
- `anyMatch(Predicate)` - Checks if any element matches.
- `noneMatch(Predicate)` - Checks if no elements match.

Finding

- `findFirst()` - Returns the first element.
- `findAny()` - Returns any element.

Example:

```

import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class TerminalOps {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(1, 2, 3);
        // 1. collect
        list.stream().skip(1).collect(Collectors.toList()); // Skips the first element and collects: [2, 3]
        list.stream().skip(1).toList(); // Skips the first element and collects: [2, 3]

        // 2. forEach
        list.stream().forEach(x -> System.out.println(x));
        // Output:
        // 1
        // 2
        // 3

        // 3. reduce : Combines elements to produce a single result
        Optional<Integer> optionalInteger = list.stream().reduce(Integer::sum);
        System.out.println(optionalInteger.get());
        // Output: 6 (1 + 2 + 3)

        // 4. count
        System.out.println(list.stream().count());
        // Output: 3 (number of elements in the list)

        // 5. anyMatch, allMatch, noneMatch
        boolean b = list.stream().anyMatch(x -> x % 2 == 0);
        System.out.println(b);
        // Output: true (2 is even)

        boolean b1 = list.stream().allMatch(x -> x > 0);
        System.out.println(b1);
        // Output: true (all elements are greater than 0)

        boolean b2 = list.stream().noneMatch(x -> x < 0);
        System.out.println(b2);
        // Output: true (no element is less than 0)

        // 6. findFirst, findAny
        System.out.println(list.stream().findFirst().get());
        // Output: 1 (first element in the list)

        System.out.println(list.stream().findAny().get());
        // Output: 1 (findAny can return any element, but in sequential streams, it will return the first)

        // 7. toArray()
        Object[] array = Stream.of(1, 2, 3).toArray();
        // Output: [1, 2, 3] (not printed, but the array contains these elements)
    }
}

```

```
// 8. min / max
System.out.println("max: " + Stream.of(2, 44, 69).max((o1, o2) -> o2 - o1));
// Output: max: Optional[69]

System.out.println("min: " + Stream.of(2, 44, 69).min(Comparator.naturalOrder()));
// Output: min: Optional[2]

// 9. forEachOrdered
List<Integer> numbers0 = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
System.out.println("Using forEach with parallel stream:");
numbers0.parallelStream().forEach(System.out::println);
// Output: Order may vary (parallel stream)

System.out.println("Using forEachOrdered with parallel stream:");
numbers0.parallelStream().forEachOrdered(System.out::println);
// Output: 1 2 3 4 5 6 7 8 9 10 (ordered)

// Example: Filtering and Collecting Names
List<String> names = Arrays.asList("Anna", "Bob", "Charlie", "David");
System.out.println(names.stream().filter(x -> x.length() > 3).toList());
// Output: [Charlie, David]

// Example: Squaring and Sorting Numbers
List<Integer> numbers = Arrays.asList(5, 2, 9, 1, 6);
System.out.println(numbers.stream().map(x -> x * x).sorted().toList());
// Output: [1, 4, 25, 36, 81]

// Example: Summing Values
List<Integer> integers = Arrays.asList(1, 2, 3, 4, 5);
System.out.println(integers.stream().reduce(Integer::sum).get());
// Output: 15 (sum of all elements)

// Example: Counting Occurrences of a Character
String sentence = "Hello world";
System.out.println(sentence.chars().filter(x -> x == 'l').count());
// Output: 3 (three 'l' characters in "Hello world")

// Example
// Streams cannot be reused after a terminal operation has been called
Stream<String> stream = names.stream();
stream.forEach(System.out::println);
// Output:
// Anna
// Bob
// Charlie
// David

// List<String> list1 = stream.map(String::toUpperCase).toList();
// Exception: IllegalStateException (stream has already been consumed)
}
}
```

4. Collecting Results

Basic Collectors

- `toList()` - Collects elements into a `List`.

- `toSet()` - Collects elements into a `Set`.
- `toMap(Function, Function)` - Collects elements into a `Map`.

```
numbers.stream().collect(Collectors.toList());
```

Advanced Collectors

Grouping

- `groupingBy(Function)` - Groups elements by a key.
- `groupingBy(Function, Collector)` - Applies further aggregation.

```
// Group strings by length
strings.stream().collect(Collectors.groupingBy(String::length));

// Group and count occurrences
strings.stream().collect(Collectors.groupingBy(Function.identity(), Collectors.counting
()));
```

Partitioning

- `partitioningBy(Predicate)` - Splits elements into true/false groups.

```
numbers.stream().collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

Joining

- `joining()` - Concatenates strings.
- `joining(delimiter)` - Adds delimiter.

```
strings.stream().collect(Collectors.joining(", "));
```

Summarizing

- `summarizingInt(ToIntFunction)` - Provides count, sum, min, average, and max.
- `summarizingDouble(ToDoubleFunction)` / `summarizingLong(ToLongFunction)`

```
IntSummaryStatistics stats = numbers.stream().collect(Collectors.summarizingInt(Integer::i
ntValue));
System.out.println(stats.getSum()); // Total sum
System.out.println(stats.getAverage()); // Average
```

More detailed example

```
import java.util.Arrays;
import java.util.Random;
import java.util.stream.Collectors;
import java.util.stream.DoubleStream;
import java.util.stream.IntStream;

public class PrimitiveStreams {
    public static void main(String[] args) {
        // Create an IntStream from an array of integers
        int[] numbers = {1, 2, 3, 4, 5};
        IntStream stream = Arrays.stream(numbers);
```

```

// Create a range of integers (exclusive of the end point) and collect them into a list
System.out.println(IntStream.range(1, 5).boxed().collect(Collectors.toList()));
// Output: [1, 2, 3, 4]

// Create a range of integers (inclusive of the end point) and collect them into a list
System.out.println(IntStream.rangeClosed(1, 5).boxed().collect(Collectors.toList()));
// Output: [1, 2, 3, 4, 5]

// Create an IntStream with specific values
IntStream.of(1, 2, 3);
// This creates a stream but no terminal operation is called, so no output is generated

// Create a DoubleStream of random double values (size: 5)
DoubleStream doubles = new Random().doubles(5);

// Collect the DoubleStream into a list of boxed Double values
System.out.println(doubles.boxed().toList());
// Output: [Random double values, e.g., 0.8327295695110215, 0.9923808454971016, ...]

// Create an IntStream of random integers (size: 5)
IntStream intStream = new Random().ints(5);

// Collect the IntStream into a list of boxed Integer values
System.out.println(intStream.boxed().toList());
// Output: [Random integers, e.g., -178383634, 210485765, ...]
}
}

```

Reducing

- `reducing(BinaryOperator)` - Reduces to a single value.

```
numbers.stream().collect(Collectors.reducing(0, Integer::sum)); // Sum of all elements
```

5. Grouping and Aggregation Examples

1. Group by Length of Strings:

```
Map<Integer, List<String>> grouped = strings.stream()
    .collect(Collectors.groupingBy(String::length));
```

2. Group by Even/Odd Numbers:

```
Map<Boolean, List<Integer>> partitioned = numbers.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

3. Count Occurrences of Words:

```
Map<String, Long> wordCount = strings.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

4. Sum and Average by Groups:

```
Map<Boolean, Double> averageByParity = numbers.stream()
    .collect(Collectors.partitioningBy(
        n -> n % 2 == 0,
```

```
Collectors.averagingInt(Integer::intValue)
));
```

5. Get Statistics (Sum, Min, Max, Avg):

```
IntSummaryStatistics stats = numbers.stream()
    .collect(Collectors.summarizingInt(Integer::intValue));
```

6. Custom Reduction:

```
int sum = numbers.stream().collect(Collectors.reducing(0, Integer::sum));
```

Example program:

```
import java.util.*;
import java.util.stream.Collectors;
import java.util.stream.Collectors;

public class CollectorsDemo {
    public static void main(String[] args) {
        // Collectors is a utility class that provides methods to create common collectors

        // 1. Collecting to a List
        List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
        List<String> res = names.stream()
            .filter(name -> name.startsWith("A"))
            .collect(Collectors.toList());
        System.out.println(res);
        // Output: [Alice]

        // 2. Collecting to a Set
        List<Integer> nums = Arrays.asList(1, 2, 2, 3, 4, 4, 5);
        Set<Integer> set = nums.stream().collect(Collectors.toSet());
        System.out.println(set);
        // Output: [1, 2, 3, 4, 5]

        // 3. Collecting to a Specific Collection
        ArrayDeque<String> collect = names.stream().collect(Collectors.toCollection(() ->
new ArrayDeque<>()));
        System.out.println(collect);
        // Output: [Alice, Bob, Charlie]

        // 4. Joining Strings
        String concatenatedNames = names.stream().map(String::toUpperCase).collect(Collectors.joining(", "));
        System.out.println(concatenatedNames);
        // Output: ALICE, BOB, CHARLIE

        // 5. Summarizing Data
        List<Integer> numbers = Arrays.asList(2, 3, 5, 7, 11);
        IntSummaryStatistics stats = numbers.stream().collect(Collectors.summarizingInt(x
-> x));
        System.out.println("Count: " + stats.getCount());
        System.out.println("Sum: " + stats.getSum());
        System.out.println("Min: " + stats.getMin());
        System.out.println("Average: " + stats.getAverage());
        System.out.println("Max: " + stats.getMax());
        /*

```



```

Output:
Count: 5
Sum: 28
Min: 2
Average: 5.6
Max: 11
*/

// 6. Calculating Averages
Double average = numbers.stream().collect(Collectors.averagingInt(x -> x));
System.out.println("Average: " + average);
// Output: Average: 5.6

// 7. Counting Elements
Long count = numbers.stream().collect(Collectors.counting());
System.out.println("Count: " + count);
// Output: Count: 5

// 8. Grouping Elements
List<String> words = Arrays.asList("hello", "world", "java", "streams", "collectin
g");

System.out.println(words.stream().collect(Collectors.groupingBy(String::length)));
// Output: {5=[hello, world, java], 7=[streams], 10=[collecting]}

System.out.println(words.stream().collect(Collectors.groupingBy(String::length, Co
llectors.joining(", ")))));
// Output: {5=hello, world, java, 7=streams, 10=collecting}

System.out.println(words.stream().collect(Collectors.groupingBy(String::length, Co
llectors.counting())));
// Output: {5=3, 7=1, 10=1}

TreeMap<Integer, Long> treeMap = words.stream()
    .collect(Collectors.groupingBy(String::length, TreeMap::new, Collectors.co
unting()));
System.out.println(treeMap);
// Output: {5=3, 7=1, 10=1}

// 9. Partitioning Elements
System.out.println(words.stream().collect(Collectors.partitioningBy(x -> x.length
() > 5)));
// Output: {false=[hello, world, java], true=[streams, collecting]}

// 10. Mapping and Collecting
System.out.println(words.stream().collect(Collectors.mapping(x -> x.toUpperCase(),
Collectors.toList())));
// Output: [HELLO, WORLD, JAVA, STREAMS, COLLECTING]

// 11. toMap
// Example 1: Collecting Names by Length
List<String> l1 = Arrays.asList("Anna", "Bob", "Alexander", "Brian", "Alice");
System.out.println(l1.stream().collect(Collectors.groupingBy(String::length)));
// Output: {4=[Anna, Bob, Brian, Alice], 9=[Alexander]}

// Example 2: Counting Word Occurrences
String sentence = "hello world hello java world";
System.out.println(Arrays.stream(sentence.split(" "))
    .collect(Collectors.groupingBy(x -> x, Collectors.counting())));

```

```

// Output: {hello=2, world=2, java=1}

// Example 3: Partitioning Even and Odd Numbers
List<Integer> l2 = Arrays.asList(1, 2, 3, 4, 5, 6);
System.out.println(l2.stream().collect(Collectors.partitioningBy(x -> x % 2 ==
0)));
// Output: {false=[1, 3, 5], true=[2, 4, 6]}

// Example 4: Summing Values in a Map
Map<String, Integer> items = new HashMap<>();
items.put("Apple", 10);
items.put("Banana", 20);
items.put("Orange", 15);
System.out.println(items.values().stream().reduce(Integer::sum));
// Output: Optional[45]
System.out.println(items.values().stream().collect(Collectors.summingInt(x ->
x)));
// Output: 45

// Example 5: Creating a Map from Stream Elements
List<String> fruits = Arrays.asList("Apple", "Banana", "Cherry");
System.out.println(fruits.stream().collect(Collectors.toMap(x -> x.toUpperCase(),
x -> x.length())));
// Output: {APPLE=5, BANANA=6, CHERRY=6}

// Example 6: Counting Word Frequencies in a List
List<String> words2 = Arrays.asList("apple", "banana", "apple", "orange", "banan
a", "apple");
System.out.println(words2.stream().collect(Collectors.toMap(k -> k, v -> 1, (x, y)
-> x + y)));
// Output: {orange=1, banana=2, apple=3}
}
}

```

6. Parallel vs Sequential

- **Sequential:** `stream()`
- **Parallel:** `parallelStream()`, `parallel()`

ParallelStream

Parallel streams is used for processing large datasets, comparing their performance to sequential streams.

1. Introduction to Parallel Streams

- **Definition:** A parallel stream enables processing elements in parallel using multiple threads.
- **Advantages:**
 - Significant performance improvement for large datasets or CPU-intensive tasks.
 - Efficient workload distribution across threads.

2. Performance Comparison: Sequential vs. Parallel Streams

```

long startTime = System.currentTimeMillis();
List<Integer> list = Stream.iterate(1, x -> x + 1).limit(20000).toList();
List<Long> factorialsList = list.stream().map(ParallelStream::factorial).toList();

```

```
long endTime = System.currentTimeMillis();
System.out.println("Time taken with sequential stream: " + (endTime - startTime) + " ms");
```

- **Steps:**
 1. Create a list of integers using `Stream.iterate()`.
 2. Use a sequential stream to calculate the factorial for each integer in the list.
 3. Measure the execution time using `System.currentTimeMillis()`.

```
startTime = System.currentTimeMillis();
factorialsList = list.parallelStream().map(ParallelStream::factorial).toList();
endTime = System.currentTimeMillis();
System.out.println("Time taken with parallel stream: " + (endTime - startTime) + " ms");
```

- **Steps:**
 1. Use a parallel stream to calculate the factorial for each integer in the list.
 2. Compare execution time with the sequential stream.

3. Cumulative Sum Calculation

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
AtomicInteger sum = new AtomicInteger(0);
List<Integer> cumulativeSum = numbers.stream().sequential().map(sum::addAndGet).toList();
```

- **Goal:** Compute the cumulative sum of a list of numbers.
- **Logic:**
 - Use `AtomicInteger` to maintain thread-safe state during calculations.
 - Use a sequential stream to apply `addAndGet` for cumulative addition.
- **Example:**
 - Input: `[1, 2, 3, 4, 5]`
 - Expected Output: `[1, 3, 6, 10, 15]`

4. Factorial Function

```
private static long factorial(int n) {
    long result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

- **Purpose:** Calculate the factorial of a given number `n`.
- **Algorithm:**
 - Initialize `result` to `1`.
 - Multiply `result` by every integer from `2` to `n`.

5. Key Observations

1. **Sequential Streams:**
 - Suitable for small datasets or simple tasks.

- No threading overhead.

2. Parallel Streams:

- Most effective for:
 - Large datasets.
 - CPU-intensive tasks.
- Less efficient for small tasks due to thread management overhead.

3. Caution:

- Parallel streams may produce unexpected results if tasks are not independent.
- Example: The cumulative sum task requires sequential order; parallel processing could lead to incorrect results.

6. Output Example

• Sequential Stream Execution Time:

Time taken with sequential stream: 150 ms

• Parallel Stream Execution Time:

Time taken with parallel stream: 90 ms

• Cumulative Sum:

Expected cumulative sum: [1, 3, 6, 10, 15]
Actual result with parallel stream: [1, 3, 6, 10, 15]

This output highlights the performance boost from parallel streams for factorial computation while maintaining correctness for the cumulative sum in sequential mode.

Let me know if you need any further clarifications!

Common Patterns for Interview Questions

1. Palindrome Check:

```
strings.stream()
    .filter(s -> s.equals(new StringBuilder(s).reverse().toString()))
    .collect(Collectors.toList());
```

2. Find First Non-Empty String:

```
strings.stream().filter(s -> !s.isEmpty()).findFirst().orElse("No match");
```

3. Reverse a List Preserving Order:

```
Collections.reverse(numbers);
```

4. Calculate Average Length of Strings:

```
double avg = strings.stream()
    .collect(Collectors.averagingInt(String::length));
```

5. Flatten Nested Lists:

```
nestedLists.stream()
    .flatMap(Collection::stream)
```



```
.collect(Collectors.toList());
```

4. Method References

- Shorthand for lambda expressions that invoke a method.

- **Syntax:**

- Static Method: `ClassName::methodName`
- Instance Method: `instance::methodName`
- Constructor: `ClassName::new`

- **Examples:**

```
List<String> names = Arrays.asList("John", "Jane", "Jack");

// Using lambda
names.forEach(name -> System.out.println(name));

// Using method reference
names.forEach(System.out::println);
-----

// Another example
import java.util.function.*;

public class MethodReferenceDemo {
    // Static method
    public static void staticMethod(String s) {
        System.out.println("Static: " + s);
    }

    // Instance method
    public void instanceMethod(String s) {
        System.out.println("Instance: " + s);
    }

    public static void main(String[] args) {
        // 1. Reference to a static method
        Consumer<String> staticRef = MethodReferenceDemo::staticMethod;
        staticRef.accept("Static Reference");

        // 2. Reference to an instance method of a particular object
        MethodReferenceDemo obj = new MethodReferenceDemo();
        Consumer<String> instanceRef = obj::instanceMethod;
        instanceRef.accept("Instance Reference");

        // 3. Reference to an instance method of an arbitrary object
        Function<String, String> toUpperCase = String::toUpperCase;
        System.out.println(toUpperCase.apply("arbitrary reference"));

        // 4. Reference to a constructor
        Supplier<MethodReferenceDemo> constructorRef = MethodReferenceDemo::new;
        MethodReferenceDemo newInstance = constructorRef.get();
    }
}
```

5. Default and Static Methods in Interfaces

- **Default Methods:**

Allow interfaces to have method implementations.

```
interface MyInterface {
    default void display() {
        System.out.println("Default Method");
    }
}
```

- **Static Methods:**

Can be accessed using the interface name.

```
interface MyInterface {
    static void print() {
        System.out.println("Static Method");
    }
}

MyInterface.print();
```

6. Optional

- `Optional` is a container object used to contain **non-null** objects.
- Avoids `NullPointerException`.

Key Methods in `Optional` Class

- `empty()`
 - Returns an empty `Optional`, representing the absence of a value.
 - Example:

```
Optional<String> empty = Optional.empty();
```

- `of(T value)`
 - Returns an `Optional` containing the specified non-null value.
 - Throws `NullPointerException` if the value is `null`.
 - Example:

```
Optional<String> name = Optional.of("John");
```

- `ofNullable(T value)`
 - Returns an `Optional` describing the given value, or an empty `Optional` if the value is `null`.
 - Example:

```
Optional<String> name = Optional.ofNullable(getName()); // can be null
```

- `isPresent()`
 - Returns `true` if the value is present, otherwise `false`.
 - Example:

```
if (optional.isPresent()) {
    System.out.println(optional.get());
}
```

```
}
```

- `ifPresent(Consumer<? super T> action)`

- Executes the given action if the value is present.
- Example:

```
optional.ifPresent(value -> System.out.println(value));
```

- `get()`

- Returns the value if present, otherwise throws `NoSuchElementException`.
- Should be used cautiously as it can throw an exception if the value is absent.
- Example:

```
String value = optional.get(); // Use only if sure the value is present
```

- `orElse(T other)`

- Returns the value if present, otherwise returns the specified default value.
- Example:

```
String value = optional.orElse("Default Value");
```

- `orElseGet(Supplier<? extends T> other)`

- Returns the value if present, otherwise invokes the supplied `Supplier` to return a default value.
- Example:

```
String value = optional.orElseGet(() -> "Generated Default Value");
```

- `orElseThrow(Supplier<? extends X> exceptionSupplier)`

- Returns the value if present, otherwise throws an exception provided by the supplier.
- Example:

```
String value = optional.orElseThrow(() -> new IllegalArgumentException("Value is missin  
g"));
```

- `map(Function<? super T, ? extends U> mapper)`

- If the value is present, it applies the given function to it and returns an `Optional` of the result.
- Example:

```
Optional<String> upperCaseName = optional.map(String::toUpperCase);
```

- `flatMap(Function<? super T, Optional<U>> mapper)`

- Similar to `map()`, but the function must return an `Optional`.
- Example:

```
Optional<String> upperCaseName = optional.flatMap(name -> Optional.of(name.toUpperCase  
()));
```

- `filter(Predicate<? super T> predicate)`

- Returns the `Optional` if the value is present and satisfies the predicate, otherwise returns an empty `Optional`.
- Example:

```
Optional<String> filtered = optional.filter(name -> name.length() > 3);
```

Optional Use Cases

- **Avoiding `NullPointerException`:** Instead of directly checking for null, `Optional` allows for safer handling.

```
Optional<String> name = Optional.ofNullable(getName());  
name.ifPresent(n -> System.out.println("Name is: " + n));
```

- **Chaining methods:** With methods like `map()`, `flatMap()`, and `filter()`, you can perform operations only if a value is present, making the code more functional.

```
Optional<String> name = Optional.ofNullable("John");  
name.filter(n -> n.length() > 3).map(String::toUpperCase).ifPresent(System.out::println);
```

- **Returning `Optional` values from methods:** Methods can return an `Optional` to represent a value that may or may not be present.

```
public Optional<String> findNameById(int id) {  
    // some logic to find name, can return Optional.empty() if not found  
}
```

Best Practices

- **Use `Optional` as a return type, not as a field:** Avoid using `Optional` for fields or parameters; it's meant to be used for return types where the absence of a value is meaningful.
- **Avoid using `get()` unless necessary:** Instead, prefer methods like `ifPresent()`, `orElse()`, or `orElseThrow()` to handle empty cases more safely.
- **Use `Optional` to represent the possibility of no value,** but don't overuse it. It's not a catch-all solution for every case.

5. Example Usage

```
import java.util.Optional;  
  
public class OptionalExample {  
    public static void main(String[] args) {  
        Optional<String> name = Optional.ofNullable("John");  
  
        // If value is present, print it  
        name.ifPresent(n -> System.out.println("Name: " + n));  
  
        // Get value or return default if absent  
        String value = name.orElse("Default Name");  
        System.out.println(value);  
  
        // Throw exception if value is absent  
        String result = name.orElseThrow(() -> new IllegalArgumentException("Name is missin  
g"));  
  
        // Map the value if present  
        Optional<String> upperCaseName = name.map(String::toUpperCase);  
        upperCaseName.ifPresent(System.out::println);  
    }  
}
```

Summary

The `Optional` class in Java 8 is a powerful tool for handling null values more safely and functionally. It helps to:

- Avoid null checks and `NullPointerException`.
- Chain operations on values.
- Return meaningful results when a value might be absent.

Let me know if you need more details or examples!

- **Methods in `Optional` :**

Method	Description
<code>of(value)</code>	Creates an <code>Optional</code> for non-null value.
<code>ofNullable(value)</code>	Accepts null values.
<code>isPresent()</code>	Checks if value is present.
<code>orElse(value)</code>	Returns value or default.
<code>ifPresent()</code>	Executes if value is present.

- **Examples:**

```
Optional<String> name = Optional.ofNullable(null);

// Default value if null
System.out.println(name.orElse("Default Name"));

// Perform action if value is present
name.ifPresent(System.out::println);
```

7. Date and Time API

- Replaces `java.util.Date` and `Calendar` with `java.time`.

- **Key Classes:**

Class	Description
<code>LocalDate</code>	Represents a date.
<code>LocalTime</code>	Represents a time.
<code>LocalDateTime</code>	Combines date and time.
<code>ZonedDateTime</code>	Handles time zones.

- **Examples:**

```
LocalDate today = LocalDate.now();
System.out.println(today);

// ...

LocalDate nextWeek = today.plusWeeks(1);
System.out.println(nextWeek);
```

10. Enhancements in `java.util.Map`

- **New Methods:**

```
map.putIfAbsent(key, value);
map.forEach((k, v) -> System.out.println(k + ": " + v));
map.computeIfAbsent(key, k -> newValue);
map.computeIfPresent(key, (k, v) -> v + newValue);
```

Key Notes for Interviews

- **Why Java 8?**
 - Enhances functional programming with lambdas and streams.
 - Improved performance with parallel streams.
 - Replaces older, verbose APIs (like `Date`) with modern ones.
- **Common Interview Questions:**
 1. **Explain Lambda Expressions and Functional Interfaces.**
 2. **What is the difference between `map` and `flatMap` ?**
 3. **How does `Optional` help in avoiding `NullPointerException` ?**
 4. **What are default and static methods in interfaces?**
 5. **How do method references work?**

Ways to create Streams

Parallel Stream

Java 8 Collection Improvements